Trabajo Práctico: Análisis y Validación de Algoritmos de Optimización

Jamil Cahuana[1]

[1]Universidad Nacional de San Agustín
Arequipa - Perú

jcahuanas@unsa.edu.pe

Abstract. *Numerical optimization plays a central role in many areas of science and engineering, particularly in machine learning and data analysis. In this work, we study the behavior of several first-order optimization methods, including Gradient Descent, Stochastic Gradient Descent, Mini-batch SGD, Nesterov and Adam, applied to quadratic cost functions defined by different matrix configurations.*

The experiments analyze how properties such as convexity, strong convexity, ill-conditioning and non-convexity affect convergence, stability and computational efficiency. Additionally, Singular Value Decomposition (SVD) is employed to study the spectral structure of matrices and its impact on optimization performance and image reconstruction.

The results show that strong convexity ensures fast and stable convergence, while ill-conditioned and non-convex problems significantly hinder optimization. These findings highlight the importance of understanding the mathematical structure of the problem when selecting optimization algorithms and hyperparameters.

Resumo. *La optimización numérica desempeña un papel fundamental en muchas áreas de la ciencia y la ingeniería, en particular en el aprendizaje automático y el análisis de datos. En este trabajo, estudiamos el comportamiento de varios métodos de optimización de primer orden, como el Descenso de Gradiente, el Descenso de Gradiente Estocástico, la Descomposición de Graduados de Minilotes (SGD), Nesterov y Adam, aplicados a funciones de coste cuadráticas definidas por diferentes configuraciones matriciales.*

Los experimentos analizan cómo propiedades como la convexidad, la convexidad fuerte, el mal condicionamiento y la no convexidad afectan la convergencia, la estabilidad y la eficiencia computacional. Además, se emplea la Descomposición en Valores Singulares (SVD) para estudiar la estructura espectral de las matrices y su impacto en el rendimiento de la optimización y la reconstrucción de imágenes.

Los resultados muestran que la convexidad fuerte garantiza una convergencia rápida y estable, mientras que los problemas mal condicionamiento y no convexos dificultan significativamente la optimización. Estos hallazgos resaltan la importancia de comprender la estructura matemática del problema al seleccionar algoritmos de optimización e hiperparámetros.

1. Introducción

La optimización numérica constituye un pilar fundamental en múltiples áreas de la ciencia y la ingeniería, particularmente en aprendizaje automático, análisis de datos y procesamiento de señales. Muchos de estos problemas pueden formularse como la minimización de funciones de costo cuadráticas, cuya estructura depende directamente de las propiedades de una matriz asociada, típicamente el Hessiano del problema.

En este trabajo se estudia el comportamiento de distintos métodos de optimización de primer orden —incluyendo Gradient Descent (GD), Stochastic Gradient Descent (SGD), Mini-batch SGD, Nesterov y Adam— aplicados a funciones cuadráticas definidas por diferentes configuraciones de la matriz A. El objetivo principal es analizar cómo propiedades como la convexidad, la fuerte convexidad, el condicionamiento numérico y la no convexidad influyen en la convergencia, estabilidad y eficiencia computacional de los algoritmos.

Adicionalmente, se incorpora el uso de la Descomposición en Valores Singulares (SVD) como herramienta de análisis espectral y reducción de dimensionalidad, ilustrando cómo la estructura matricial impacta tanto en problemas de optimización como en aplicaciones prácticas como la compresión de imágenes.

El estudio combina fundamentos teóricos, implementación computacional en C++ y análisis experimental, permitiendo una comparación sistemática del desempeño de los métodos bajo distintos escenarios matemáticos y parámetros de aprendizaje.

2. Metodología

La metodología empleada posee un enfoque teórico-experimental. Ya que se analizan las propiedades matemáticas de las funciones empleadas en el proyecto, en particular la convexidad, la condición numérica y la estructura espectral de las matrices empleadas. Luego se trabaja con los algoritmos de optimización basados en gradiente y sus variantes y se evalua su desempeño mediante experimentación computacional. Y finalmente, se comparan métricas de convergencia, estabilidad, sensibilidad a la tasa de aprendizaje y tiempo de ejecución para las configuraciones.

3. Configuraciones de la Matriz A empleadas en las pruebas

Con el objetivo de analizar el comportamiento de distintos métodos de optimización bajo diferentes propiedades matemáticas de la función objetivo, se consideraron varias configuraciones de la matriz A. En todos los casos se estudió la función cuadrática de la forma:

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x,$$

cuyas propiedades de convexidad y condición numérica dependen directamente de la matriz A.

3.1. Matriz que induce fuerte convexidad

Se utilizó como configuración inicial la siguiente matriz:

$$A = \begin{pmatrix} 4 & 1 & 2 \\ 1 & 3 & 1 \\ 2 & 1 & 5 \end{pmatrix}$$

Justificación. La matriz A es simétrica y definida positiva, ya que todos sus autovalores son estrictamente mayores que cero. En consecuencia, la función objetivo es *fuertemente convexa*, lo que garantiza la existencia de un único mínimo global y asegura convergencia lineal para métodos basados en gradiente bajo condiciones adecuadas sobre la tasa de aprendizaje.

3.2. Matriz convexa pero no fuertemente convexa

Para analizar el caso convexo sin fuerte convexidad, se consideró:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Justificación. Los autovalores de esta matriz son $\{1, 1, 0\}$, lo que implica que A es semidefinida positiva. Por lo tanto, la función objetivo es convexa, pero no estrictamente convexa. En este caso existen infinitos mínimos globales, lo cual puede ralentizar la convergencia y generar trayectorias planas en el espacio de soluciones.

3.3. Matriz mal condicionada

Con el fin de estudiar el impacto del condicionamiento numérico, se empleó la siguiente matriz:

$$A = \begin{pmatrix} 1000 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.001 \end{pmatrix}.$$

Justificación. Esta matriz es definida positiva, pero presenta un número de condición elevado debido a la gran disparidad entre sus autovalores. Como consecuencia, la función objetivo, aunque convexa, es mal condicionada, lo que dificulta la convergencia de los métodos de optimización y suele requerir un número considerablemente mayor de iteraciones.

3.4. Matriz no convexa

Finalmente, para analizar el comportamiento de los métodos en escenarios sin convexidad, se consideró:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{pmatrix}.$$

Justificación. La matriz presenta autovalores de distinto signo, lo que implica que el Hessiano de la función es indefinido. En consecuencia, la función objetivo es no convexa, perdiéndose las garantías teóricas de convergencia. En este caso, los métodos pueden divergir o estancarse en puntos silla, dependiendo de la inicialización y la tasa de aprendizaje.

4. Algoritmos empleados en el proyecto

Se empleó y comparó los siguientes algoritmos ya implementados en el proyecto, por ello el análisis no parte de una implementación desde cero sino una comparativa entre estos. GD [Cauchy 1847] SGD Robbins [Robbins and Monro 1951] Mini batch SGD[Bottou 2010] Nesterov [Nesterov 1983] Adam[Kingma and Ba 2015]

4.1. Gradient Descent (GD)

El descenso por gradiente es un método determinista para resolver problemas de optimización de la forma

$$\min_{x \in R^n} f(x),$$

donde f es diferenciable. En aprendizaje automático, f suele corresponder al riesgo empírico

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} \ell(x; y_i, \hat{y}_i).$$

La actualización iterativa se define como

$$x_{k+1} = x_k - \alpha \nabla f(x_k).$$

Para funciones fuertemente convexas con gradiente Lipschitz continuo, GD converge linealmente.

4.2. Stochastic Gradient Descent (SGD)

El SGD utiliza un gradiente estocástico basado en una única muestra,

$$x_{k+1} = x_k - \alpha \nabla \ell(x_k; y_i, \hat{y}_i),$$

el cual es un estimador insesgado del gradiente real. Aunque introduce ruido, permite optimización eficiente a gran escala y converge en esperanza.

4.3. Mini-batch Stochastic Gradient Descent

El método mini-batch aproxima el gradiente mediante subconjuntos de datos:

$$x_{k+1} = x_k - \alpha \frac{1}{m} \sum_{i \in \mathcal{B}} \nabla \ell(x_k; y_i, \hat{y}_i),$$

reduciendo la varianza del gradiente y permitiendo paralelización computacional.

4.4. Nesterov Accelerated Gradient (NAG)

NAG introduce un término de momento anticipado,

$$v_{k+1} = \mu v_k - \alpha \nabla f(x_k + \mu v_k),$$

logrando una tasa de convergencia $\mathcal{O}(1/k^2)$ para funciones convexas suaves.

4.5. Adam (Adaptive Moment Estimation)

Adam combina momento y escalamiento adaptativo del gradiente,

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2,$$

siendo ampliamente utilizado en problemas no convexos de aprendizaje profundo.

4.6. k-Nearest Neighbors (k-NN)

k-NN es un método no paramétrico que aproxima la función de decisión mediante la minimización implícita de distancias euclidianas en el espacio de características.

4.7. Singular Value Decomposition (SVD)

La descomposición en valores singulares factoriza una matriz como $A = U\Sigma V^T$ y permite obtener aproximaciones de rango reducido óptimas en norma de Frobenius.

4.8. Automatic Differentiation (AutoDiff)

La diferenciación automática evalúa derivadas exactas mediante grafos computacionales y la aplicación sistemática de la regla de la cadena, siendo esencial en optimización moderna.

5. Funciones de BLAS y LAPACK empleadas en el proyecto

En esta sección se describen las funcións de las bibliotecas BLAS y LAPACK empleadas en distintos de los algoritmos de procesamiento matricial en el proyecto.

5.1. Función `cblas_dgemv`

La función `cblas_dgemv` pertenece al nivel 2 de BLAS y se utiliza para realizar la multiplicación matriz–vector. Matemáticamente, la operación que realiza es:

$$\mathbf{y} \leftarrow \alpha \mathbf{A}\mathbf{x} + \beta \mathbf{y} \tag{1}$$

Donde $\mathbf{A} \in R^{m\times n}$ es una matriz, $\mathbf{x} \in R^n$ y $\mathbf{y} \in R^m$ son vectores, y α, β son escalares.

En los archivos `Adam.cpp`, `autodiff.cpp`, `Nesterov.cpp` y `GradientDescent.cpp`, esta función se emplea para calcular productos del tipo $\mathbf{Ax}$, los cuales aparecen en el cómputo de gradientes, Jacobianos y evaluaciones lineales del modelo durante los métodos de optimización.

Este es un entendimiento de la función a partir de lo consultado en la bibliografía [Netlib BLAS 2023d].

5.2. Función `cblas_daxpy`

La función `cblas_daxpy` pertenece al nivel 1 de BLAS y realiza una operación vectorial elemental definida como:

$$\mathbf{y} \leftarrow \alpha \mathbf{x} + \mathbf{y} \tag{2}$$

En `GradientDescent.cpp`, esta función se utiliza para actualizar el vector de parámetros x mediante el descenso por gradiente:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k) \tag{3}$$

En `MiniBatchSGD.cpp`, la actualización se realiza usando el gradiente promedio del mini–batch, mientras que en `SGD.cpp` se emplea el gradiente correspondiente a una única muestra seleccionada aleatoriamente.

Este es un entendimiento de la función a partir de lo consultado en la bibliografía [Netlib BLAS 2023a].

5.3. Función `cblas_dnrm2`

La función `cblas_dnrm2` calcula la norma euclídea (norma L_2) de un vector:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} \tag{4}$$

En los archivos `MiniBatchSGD.cpp`, `SGD.cpp` y `Adam.cpp`, esta función se emplea como criterio de convergencia, verificando si la norma del gradiente es menor que un umbral ε:

$$\|\nabla f(\mathbf{x})\|_2 < \varepsilon \tag{5}$$

Este es un entendimiento de la función a partir de lo consultado en la bibliografía [Netlib BLAS 2023c] [Apple Inc. 2024b].

5.4. Función `cblas_ddot`

La función `cblas_ddot` pertenece al nivel 1 de BLAS y calcula el producto punto entre dos vectores:

$$\mathbf{x}^T\mathbf{y} = \sum_{i=1}^{n} x_i y_i \tag{6}$$

En `SGD.cpp`, esta función se utiliza para calcular el producto $\mathbf{Ax}$ considerando únicamente una fila i de la matriz $\mathbf{A}$:

$$(\mathbf{Ax})_i = \mathbf{A}_{i,:} \cdot \mathbf{x} \tag{7}$$

Este enfoque es característico de métodos de optimización estocásticos que operan sobre muestras individuales.

Este es un entendimiento de la función a partir de lo consultado en la bibliografía [Netlib BLAS 2023b] [Apple Inc. 2024a].

5.5. Función `LAPACKE_dgesvd`

La función `LAPACKE_dgesvd` pertenece a la biblioteca LAPACK y se utiliza para calcular la descomposición en valores singulares (SVD) de una matriz:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{8}$$

Donde $\mathbf{U}$ y $\mathbf{V}$ son matrices ortogonales y $\mathbf{\Sigma}$ es una matriz diagonal que contiene los valores singulares.

En `SVDImage.cpp`, esta descomposición se emplea para la compresión y reconstrucción de imágenes, utilizando únicamente los valores singulares más significativos.

Este es un entendimiento de la función a partir de lo consultado en la bibliografía [LAPACK Developers 2020].

A continuación se muestra las funciones anteriormente descritas, su tipo y el uso principal que se les da en el proyecto.

Función	Tipo	Uso principal
dgemv	BLAS 2 (matriz-vector)	Producto matriz-vector
daxpy	BLAS 1 (vector-vector)	Actualización de parámetros
dnrm2	BLAS 1 (vector-vector)	Norma del gradiente
ddot	BLAS 1 (vector-vector)	Producto fila-vector
dgesvd	LAPACK	SVD para compresión/análisis

Figure 1. Funciones de CBLAS y LAPACK empleadas, Tipo y Uso. Elaboración propia.

6. Resultados

MATRIZ A = {{4, 1, 2}, {1, 3, 1}, {2, 1, 5}}; Taza de aprendizaje: 0.01

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	334	543.375	X: -0.0588238 -0.294118 0.764706
MINI BATCH SGD	743	110.104	X: -0.16279 0.534883 0.558139
SGD	794	245.417	X: -0.16279 0.534883 0.558139
ADAM	-	8.990.967	x: -26.5625 -126.731 116.99
GD	-	157.666	x: 3.87188 -42.1022 13.8804

Figure 2. Comparación de resultados de la matriz A con cinco métodos (Nesterov, Mini-batch SGD, SGD, Adam y GD) y un learning rate de 0.01

MATRIZ A = {{4, 1, 2}, {1, 3, 1}, {2, 1, 5}}; Taza de aprendizaje: 0.1

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	5379	129.388	x: 3 -37 13
MINI BATCH SGD	808	903.083	x: -0.16279 0.534884 0.558139
SGD	776	21.725	x: -0.16279 0.534883 0.558139
ADAM	-	100.726	x: -305.662 -1258.22 1240.93
GD	890	17.675	x: -0.0588233

Figure 3. Comparación de resultados de la matriz A con cinco métodos (Nesterov, Mini-batch SGD, SGD, Adam y GD) y un learning rate de 0.1

MATRIZ A = {{4, 1, 2}, {1, 3, 1}, {2, 1, 5}}; taza de aprendizaje: 0.2

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	5379	111.571	x: 3 -37 13
MINI BATCH SGD	623	947.917	x: -0.162784 0.53488 0.558138
SGD	-	230.854	x: nan nan nan
ADAM	-	993.196	x: -305.662 -1258.22 1240.93
GD	40	1	x: -0.0588233 -0.294118 0.764706

Figure 4. Comparación de resultados de la matriz A con cinco métodos (Nesterov, Mini-batch SGD, SGD, Adam y GD) y un learning rate de 0.2

MATRIZ A = {{4, 1, 2}, {1, 3, 1}, {2, 1, 5}}; taza de aprendizaje: 0.2

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	5379	111.571	x: 3 -37 13
MINI BATCH SGD	623	947.917	x: -0.162784 0.53488 0.558138
SGD	-	230.854	x: nan nan nan
ADAM	-	993.196	x: -305.662 -1258.22 1240.93
GD	40	1	x: -0.0588233 -0.294118 0.764706

Figure 5. Comparación de resultados de la matriz A con el mismo método, NESTEROV, variando un hiperparámetro; momentos 0.3, 0.6 y 0.9

MATRIZ A = {{4, 1, 2}, {1, 3, 1}, {2, 1, 5}}; taza de aprendizaje: 0.2

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	5379	111.571	x: 3 -37 13
MINI BATCH SGD	623	947.917	x: -0.162784 0.53488 0.558138
SGD	-	230.854	x: nan nan nan
ADAM	-	993.196	x: -305.662 -1258.22 1240.93
GD	40	1	x: -0.0588233 -0.294118 0.764706

Figure 6. Comparación de resultados de la matriz A, FUERTEMENTE CONVEXA, con cinco métodos (Nesterov, Mini-batch SGD, SGD, Adam y GD) y un learning rate de 0.01

MATRIZ CONVEXA, pero NO FUERTEMENTE CONVEXA A = {{1, 0, 0}, {0, 1, 0}, {0, 0, 0}}; Taza de aprendizaje: 0.01

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	5379	686.459	x: 3 -37 13
MINI BATCH SGD	783	996.958	x: -0.16279 0.534883 0.558139
SGD	-	232.508	x: nan nan nan
ADAM	-	904.938	x: 1 -123.174 100
GD	40	109542	x: 3 -37 13

Figure 7. Comparación de resultados de la matriz A, CONVEXA pero no FUERTEMENTE CONVEXA, con cinco métodos (Nesterov, Mini-batch SGD, SGD, Adam y GD) y un learning rate de 0.01

MATRIZ MAL CONDICIONADA A = {{1000, 0, 0}, {0, 1, 0}, {0, 0, 0.001}}; Taza de aprendizaje: 0.01

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	-	195.317	x: nan nan nan
MINI BATCH SGD	-	119.529	x: nan nan nan
SGD	-	169.112	x: nan nan nan
ADAM	-	998.258	x: 0.001 -123.174 100
GD	-	159459	x: nan nan nan

Figure 8. Comparación de resultados de la matriz A, MAL CONDICIONADA, con cinco métodos (Nesterov, Mini-batch SGD, SGD, Adam y GD) y un learning rate de 0.01

MATRIZ NO CONVEXA A = {{1, 0, 0}, {0, -1, 0}, {1, 0, 0}}; Taza de aprendizaje: 0.01

MÉTODO	Convergencia Alcanzada en iteración	Tiempo de ejecución (s)	Solución óptima
NESTEROV	-	209.513	x: nan nan nan
MINI BATCH SGD	-	848.167	x: nan nan nan
SGD	-	230.713	x: nan nan nan
ADAM	-	790.787	x: 1 122.648 100
GD	-	126792	x: nan nan nan

Figure 9. Comparación de resultados de la matriz A, NO CONVEXA, con cinco métodos (Nesterov, Mini-batch SGD, SGD, Adam y GD) y un learning rate de 0.01

Caso	Convexidad	Iteraciones	Estabilidad
Convexa	Sí	Media	Media
Fuertemente convexa	Sí	Pocas	Alta
Mal condicionada	Sí	Muchas	Baja
No convexa	No	No converge	Inestable

Figure 10. Comparación de las 4 matrices convexidad, iteraciones y estabilidad

Análisis de Resultados

Las Figuras 2 a 10 presentan la comparación del proceso experimental de distintos métodos de optimización basados en gradiente bajo diferentes configuraciones de la matriz A, considerando variaciones en la tasa de aprendizaje y en las propiedades de convexidad de la función objetivo. El análisis se centra en tres métricas principales: número de iteraciones hasta la convergencia, tiempo de ejecución y estabilidad de la solución.

Efecto de la tasa de aprendizaje

Los resultados muestran que para valores pequeños de la tasa de aprendizaje ($\alpha = 0.01$), los métodos presentan una convergencia estable, aunque con un número elevado de iteraciones. Al incrementar α a valores intermedios ($\alpha = 0.1$), algunos métodos aceleran la convergencia, mientras que otros comienzan a mostrar inestabilidad. Para valores grandes ($\alpha = 0.2$), varios métodos divergen o producen valores no numéricos, evidenciando la sensibilidad del proceso de optimización a la elección del paso de aprendizaje.

Comparación entre métodos

El descenso por gradiente clásico destaca por su estabilidad, aunque no siempre alcanza la convergencia más rápida. El método de Nesterov reduce significativamente el número de iteraciones gracias al uso de momento, mientras que los métodos estocásticos introducen variabilidad en la convergencia. Adam muestra buen desempeño en algunos escenarios, pero pierde estabilidad en funciones mal condicionadas o no convexas.

Influencia de la convexidad y condición numérica

En el caso de matrices fuertemente convexas, todos los métodos logran converger, aunque con diferentes velocidades. Para matrices convexas pero no fuertemente convexas, la convergencia es más lenta y menos estable. En matrices mal condicionadas, a pesar de ser convexas, la convergencia se degrada considerablemente, requiriendo más iteraciones y mostrando mayor sensibilidad a la tasa de aprendizaje. Finalmente, en el caso no convexo,

se observa la pérdida de garantías teóricas, con ausencia de convergencia en la mayoría de los métodos evaluados.

Figure 11. Las siguientes imágenes son de pruebas ejecutadas en el programa $svd_project$ configurando los valores a conservar: 100, 80, 60, 40, 30, 20, 10, 5, 3 y 2.

Análisis de la aproximación mediante SVD

La Figura correspondiente muestra los resultados obtenidos al aplicar una descomposición en valores singulares (SVD) truncada a una imagen en escala de grises, conservando un número decreciente de valores singulares: 100, 80, 60, 40, 30, 20, 10, 5, 3 y 2. Cada imagen representa una aproximación de bajo rango de la imagen original, reconstruida utilizando únicamente los k valores singulares más significativos.

Para valores altos de k (100, 80 y 60), la reconstrucción preserva la mayor parte de la información visual, incluyendo bordes, contrastes y detalles finos. Esto se debe a que los valores singulares dominantes concentran la mayor parte de la energía de la imagen, capturando las estructuras globales más relevantes.

A medida que el número de valores singulares conservados disminuye (40, 30 y 20), se observa una pérdida progresiva de detalles de alta frecuencia, especialmente en las texturas y contornos más finos, aunque la estructura general de la imagen sigue siendo

reconocible. Este comportamiento refleja el compromiso entre compresión y fidelidad, característico de las aproximaciones de bajo rango.

Cuando el número de valores singulares es muy reducido (10, 5, 3 y 2), la imagen reconstruida presenta un alto nivel de suavizado y distorsión, perdiéndose la mayoría de los detalles locales. En este régimen, la aproximación conserva únicamente las componentes más dominantes de baja frecuencia, lo que produce imágenes borrosas y con artefactos visibles.

Estos resultados confirman experimentalmente que la SVD actúa como un filtro de información jerárquico, donde los valores singulares mayores codifican la información más relevante de la imagen. Asimismo, evidencian la utilidad de la SVD truncada como técnica de compresión y reducción de dimensionalidad, así como sus limitaciones cuando se emplean rangos excesivamente bajos.

7. Resultados

Los experimentos realizados muestran diferencias significativas en el comportamiento de los métodos de optimización según la configuración de la matriz A y la tasa de aprendizaje empleada. En el caso de matrices fuertemente convexas, la mayoría de los métodos converge de manera estable hacia un único mínimo global, aunque con diferencias notables en el número de iteraciones y el tiempo de ejecución.

Al considerar matrices convexas pero no fuertemente convexas, se observa una convergencia más lenta y, en algunos casos, una mayor sensibilidad a la elección del criterio de parada, reflejando la presencia de múltiples soluciones óptimas.

En escenarios de matrices mal condicionadas, los métodos presentan dificultades importantes: el número de iteraciones aumenta considerablemente, la estabilidad numérica se reduce y algunos algoritmos divergen o producen valores indefinidos. Este comportamiento es consistente con el alto número de condición de la matriz, que amplifica los errores en la dirección del gradiente.

Finalmente, para matrices no convexas, ninguno de los métodos garantiza convergencia global. Los resultados muestran comportamientos erráticos, estancamiento en puntos silla o divergencia, confirmando las limitaciones teóricas de los métodos de primer orden en este tipo de problemas.

8. Discusión General

Este trabajo pone de manifiesto la estrecha relación entre las propiedades algebraicas de una matriz y el comportamiento dinámico de los algoritmos de optimización. La convexidad y el condicionamiento no solo determinan la existencia y unicidad de soluciones, sino que también influyen directamente en la velocidad de convergencia, la estabilidad numérica y el costo computacional.

Los resultados obtenidos refuerzan la importancia de realizar un análisis previo del problema antes de seleccionar un método de optimización. En particular, tecnicas como la SVD permiten comprender la estructura espectral del sistema y anticipar posibles dificultades asociadas a valores singulares pequeños o matrices indefinidas.

Desde una perspectiva práctica, este estudio demuestra que la combinación de teoría matemática, experimentación computacional y análisis visual constituye una es-

trategia efectiva para evaluar algoritmos de optimización en escenarios reales y controlados.

9. Resultados y Discusión

El análisis experimental confirma que las propiedades espectrales de la matriz A juegan un rol determinante en el desempeño de los métodos de optimización. En particular, la fuerte convexidad —garantizada por autovalores estrictamente positivos— proporciona un entorno favorable para la convergencia rápida y estable, especialmente para métodos como Nesterov y Adam.

En contraste, las matrices mal condicionadas evidencian la importancia de una adecuada selección de la tasa de aprendizaje. Valores elevados de α aceleran la convergencia inicial, pero incrementan el riesgo de inestabilidad y divergencia, mientras que valores pequeños aseguran estabilidad a costa de un mayor número de iteraciones.

Los métodos adaptativos, como Adam, muestran ventajas en ciertos escenarios al ajustar dinámicamente la magnitud de los pasos, aunque también pueden presentar problemas de estabilidad cuando el problema está mal condicionado o es no convexo. Por su parte, GD clásico exhibe un comportamiento más predecible, pero con menor eficiencia en términos de iteraciones.

El análisis mediante SVD refuerza estas observaciones al mostrar cómo la energía de la matriz (o de la imagen) se concentra en unos pocos valores singulares dominantes. Esto explica por qué los problemas mal condicionados —asociados a valores singulares muy pequeños— resultan especialmente difíciles para los métodos de optimización basados en gradientes.

En conjunto, los resultados subrayan que no existe un método universalmente óptimo, y que la elección del algoritmo y sus hiperparámetros debe considerar explícitamente la estructura matemática del problema.

10. References

References

Apple Inc. (2024a). cblas_ddot (accelerate framework). `https://developer.apple.com/documentation/accelerate/cblas_ddot(_:_:_:_:_:)`. Retrieved 2024.

Apple Inc. (2024b). cblas_dnrm2 (accelerate framework). `https://developer.apple.com/documentation/accelerate/cblas_dnrm2(_:_:_:)`. Retrieved 2024.

Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT*, pages 177–186. Springer.

Cauchy, A.-L. (1847). Méthode générale pour la résolution des systèmes d'équations simultanées. *Comptes Rendus de l'Académie des Sciences*, 25:536–538.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations*.

LAPACK Developers (2020). Lapacke dgesvd: Singular value decomposition. https://gensoft.pasteur.fr/docs/lapack/3.9.0/d0/dee/lapacke__dgesvd_8c_af31b3cb47f7cc3b9f6541303a2968c9f.html. Retrieved 2024.

Nesterov, Y. (1983). A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27:372–376.

Netlib BLAS (2023a). Cblas daxpy: Constant times a vector plus a vector. https://www.netlib.org/lapack/explore-html/de/da0/cblas_8h_a687bbd4f7303970498d9e398a0a87595.html. Retrieved 2024.

Netlib BLAS (2023b). Cblas ddot: Dot product of two vectors. https://www.netlib.org/lapack/explore-html/de/da0/cblas_8h_ad6fbc79028f5445105a322c6d44b608c.html. Retrieved 2024.

Netlib BLAS (2023c). Cblas dnrm2: Euclidean norm of a vector. https://www.netlib.org/lapack/explore-html/de/da0/cblas_8h_a033e206ba2b673629da7ade3179c9c6f.html. Retrieved 2024.

Netlib BLAS (2023d). Dgemv: Matrix-vector multiplication. https://www.netlib.org/lapack/explore-html/d7/dda/group__gemv_ga4ac1b675072d18f902db8a310784d802.html. Retrieved 2024.

Robbins, H. and Monro, S. (1951). A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407.